



Political Patricians (patricians)

E réimeschen Zensor beobachtet N Patrizier mat bedenkleche politeschen Usiichten. All Patrizier gehéiert zu **genee enger** geheimer politescher Partei, an all Partei huet een oder méi Patrizier. Den Zensor wëll rausfannen, wéi eng Patrizier zu wat fir enger Partei gehéieren.

Fir dat z'erreechen, organiséiert den Zensor ëmmer nees Banqueten, wouhin déi N Patrizier all invitéiert sinn. Bei jiddfer Banquet kënnen sech d'Patrizier un ënnerschiddlech Dëscher setzen, an den Zensor kontrolléiert wéi wou sëtzt. E Patrizier ass *verloossen*, wa kee Patrizier vun där selwechter Partei um selwechten Dësch sëtzt.

Als de Gastgeber weess den Zensor no all Banquet, wéi eng Patrizier verloossen do souzen. Är Aufgab ass et, dem Zensor ze hëllefen, d'politesch Parteie vun de Patrizier rauszefannen, ouni dofir missen ze vill Banqueten z'organisieren.

Implementéierung

Dir musst een eenzege `.cpp` Fichier ofginn.



An den Annexe fannt Dir en Template `patricians.cpp` mat enger Beispillimplementéierung.

Dir musst déi folgend Funktioun schreiwen:

C++

```
vector<int> find_factions(int N)
```

- Den Integer N stellt d'Unzuel u Patrizier duer. D'Patrizier sinn nummeréiert vun 0 bis $N - 1$.
- D'Funktioun soll en Array P mat N Integer-Zuelen zeréckschécken, déi déi politesch Parteie vun de Patrizier duerstellen. $P[i]$ soll gläich $P[j]$ si wann (an nëmmen dann) d'Patrizier i a j zur selwechter politescher Partei gehéieren. Et gëtt keng weider Aschränkung, wéi d'Wärter vu P gewielt sinn.

Äre Programm kann déi folgend Funktioun opruffen, déi schon am Grader definéiert ass:

C++

```
vector<bool> organize_banquet(const vector<int>& Q)
```

- Den Array Q mat N Integer-Zuele stellt d'Sëtzuordnung duer: de Patrizier i sëtzt um Dësch $Q[i]$. Et gëtt keng weider Aschränkung bei den Zuelen am Q : d'Dëscher kënnen egal wéi nummeréiert sinn, an déi eenzeg relevant Informatioun ass datt wann $Q[i] = Q[j]$, da sëtzen d'Patrizier i a j um selwechten Dësch.
- D'Funktioun schéckt en Array A mat N bool-Wärter zeréck. $A[i]$ ass true wann (an nëmmen dann) de Patrizier verlosse beim Banquet sëtzt.

De Grader ass **net adaptiv** an all Opruff leeft mat deene selwechten Testfäll duerch. Dat heescht, d'politesch Parteie vun alle Patrizier sti vu viran an den Testfäll fest an hängen net dovun of, mat wat `organize_banquet` am Programm opgeruff gëtt.

Beispill-Grader

Bei den Annexe steet eng vereinfacht Versioun vum Grader mat direkter Evaluatioun, sou datt Dir Är Léisunge lokal teste kënnt. Dëse Beispillgrader liest d'Date vun `stdin`, rifft d'Funktioon `find_factions` op a schreift d'Äntwert op `stdout` an dësem Format:

Den Input besteet aus 2 Zeilen, mat:

- Zeil 1: den Integer N .
- Zeil 2: N Integer-Zuelen, déi d'politesch Parteie vun den eenzelne Patrizier beschreiwen.

Den Output besteet aus enger eenzeger Zeil, mat dem Score, deen Är Léisung am Testfall kritt huet. Wann de Score 0 ass, schreift en e kuerze Message op `stderr` mam Grond (incorrect factions given as answer, invalid query, or too many queries).

Aschränkungen

- $4 \leq N \leq 2000$.

Dir kënnt `organize_banquet` maximal 30000 Mol opruffen.

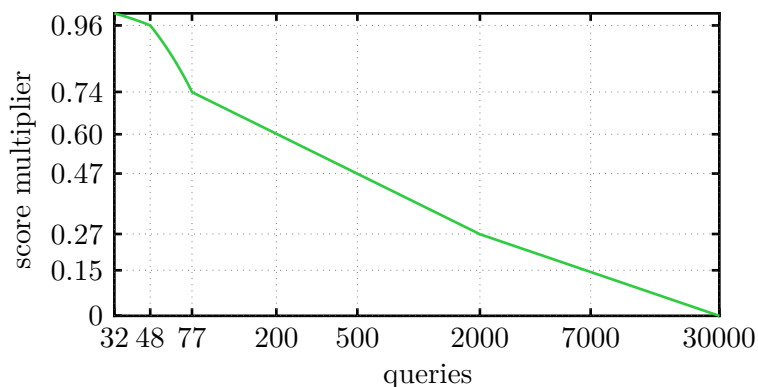
Punkteverdeelung

Äert Programm gëtt mat verschiddenen Testfäll getest, déi a Subtasks regroupéiert sinn. De Score-Multiplikator vun engem Subtask ass de Minimum vun de Score-Multiplikatoren, déi an all den Testfäll erreecht goufen.

- **Subtask 0 [0 Punkten]:** D'Beispiller aus dësem Text.
- **Subtask 1 [19 Punkten]:** Et gëtt genee zwou politesch Parteien.
- **Subtask 2 [29 Punkten]:** All politesch Partei huet genee zwee Patrizier.
- **Subtask 3 [52 Punkten]:** Keng weider Aschränkungen.

De Score-Multiplikator, deen an engem Testfall erreecht gëtt, rechent sech esou:

Unzuel u Queries	Score-Multiplikator
≤ 32	1.0
$33 \leq x < 48$	$1.08 - 0.0025x$
48	0.96
$48 < x < 77$	$1.32413 - 0.00758x$
77	0.74
$77 < x < 2000$	$1.366 - 0.1 \log_2(x)$
2000	0.27
$2000 < x \leq 30000$	$1.027 - 0.0691 \log_2(x)$



De Score vun engem Subtask ass de Minimum vun de Score-Multiplikatoren vun allen Testfäll, multiplizéiert mat dem Basis-Score vum Subtask.

Beispiller

Grader	Léisung
<code>find_factions(5)</code>	
	<code>organize_banquet([0,0,1,1,2])</code>
<code>return [1,1,1,1,1]</code>	
	<code>organize_banquet([0,0,0,1,1])</code>
<code>return [0,1,0,1,1]</code>	
	<code>organize_banquet([0,1,2,1,1])</code>
<code>return [1,0,1,0,1]</code>	
	<code>return [0,1,0,1,2]</code>

Grader	Léisung
<code>find_factions(4)</code>	
	<code>organize_banquet([1,1,0,0])</code>
<code>return [1,1,1,1]</code>	
	<code>organize_banquet([1,1,1,2])</code>
<code>return [0,1,0,1]</code>	
	<code>organize_banquet([0,1,1,1])</code>
<code>return [1,0,1,0]</code>	
	<code>return [0,1,0,1]</code>

Erklärung

Am éischte Beispill sinn d'Patrizier 0 an 2 an enger politescher Partei, d'Patrizier 1 an 3 sinn an enger anerer, an de Patrizier 4 ass an enger eegestänneger Partei. Beim éischte Banquet waren all d'Patrizier verloossen. Beim zweete Banquet waren d'Patrizier 0 an 2 net verloossen, well si souze beieneen. Beim drëtte Banquet waren d'Patrizier 1 an 3 net verloossen, well si souze beieneen.